MULTI-LABEL FOOD CLASSIFICATION USING DEEP LEARNING MODELS

Kotsonis Eleftherios, Koukovinis Panagiotis

International Hellenic University

ABSTRACT

This research investigates multi-label food recognition by training deep learning models on a broad food image dataset. The training dataset consists of 40,000 images which belong to 498 food categories and each image contains multiple labels [1]. The research evaluated three prominent convolutional neural network models which consisted of ResNet50 [2], ConvNeXt [3] and EfficientNetV2 [4]. The research attempted to boost model generalization through data augmentation methods and binary cross-entropy loss fine-tuning of pre-trained models. The highest performance results emerged from using the combination of ConvNeXt-Large with EfficientNetV2-L and EfficientNetV2-XL models. The ensemble approach achieved a public Micro F1 score of 0.50088 on the Kaggle leaderboard. The research demonstrates that using multiple models with optimized threshold values leads to successful outcomes for complex food recognition tasks that require multiple labels.

1. DATA & PROBLEM DESCRIPTION

This project draws its data from the AICrowd Food Recognition Benchmark 2022 dataset [1] which enables a multi-label image classification operation. The training dataset comprises 39,962 images which each belong to one or several of the 498 food categories. The test set contains 1,000 images that lack any label information. The labels exist in binary multi-hot encoding format which shows whether each food item appears or not in each image.

The competition demands participants to identify multiple food categories in each image because it differs from traditional single-label classification tasks. The challenge exists in dealing with complex visual features, diverse environments and large class imbalances across different food types. The identification of five or more distinct food items in images remains necessary for correct identification of each item.

The Micro F1 score functions as the main evaluation metric because it measures both precision and recall at the class and instance level. The use of micro-averaging proves suitable for this task because it addresses both the class imbalance problem and the requirement for precise label identification. The large number of labels and similar visual characteristics between food items make this task require strong feature extraction capabilities and precise prediction threshold adjustments. The project employed pre-trained deep learning models which we adapted through training on provided data using methods explained in the following section.

2. DESCRIPTION OF MODELS

We trained seven models throughout this project while making 43 submissions to enhance model architecture and training strategy based on experimental outcomes. We started with EfficientNetB0 [5] and ResNet50 [2] as baseline architectures before moving on to advanced convolutional networks including ConvNeXt [3] and EfficientNetV2 family [4]. The PyTorch framework [6] served as our implementation platform to create custom model heads and manage GPU resources while integrating augmentation and scheduling strategies. We tested the following pretrained architectures during our evaluation process:

- EfficientNetB0 [5]: Initially tested due to its efficiency, this small model underperformed due to limited depth. It was the first model we trained and did not have the proper code at the time in order to save it locally.
- ResNet50 [2]: A classic residual network, where we replaced the final linear classifier with a custom head including dropout for regularization.
- ConvNeXt-Tiny and ConvNeXt-Large [3] update ResNet architecture by implementing larger kernels, reducing inductive biases and using LayerNorm. The two-layer head (Linear → ReLU → Dropout → Linear) in ConvNeXt-Large provided enhanced learning capacity for the extensive label space.
- EfficientNetV2-M, L, and XL [4] represent state-of-the-art models that implement compound scaling and optimized training speed. The XL variant employed a two-layer MLP classifier with extra dropout layers to address multi-label classification requirements.

2.1 Training Configuration and Environment Details

The training process took place on a NVIDIA RTX 3060 GPU while the batch sizes for each model were determined by the 12 GB VRAM restrictions which limited the use of

large architectures including EfficientNetV2-XL and ConvNeXt-Large. The initial training took place in Jupyter Notebook but we encountered problems with multiprocessing compatibility. The training process became unresponsive when num_workers exceeded 0. The transition to PyCharm allowed us to use num_workers = 4 for faster data loading while maintaining stability. Due to GPU memory limitations, we adjusted our batch size depending on the model size:

- Batch sizes of 32–64 were feasible for smaller models (e.g., ResNet50, EfficientNetV2-M).
- For ConvNeXt-Large and EfficientNetV2-XL, batch size had to be reduced to 16 to avoid out-of-memory errors.

The Adam optimizer [7] served as the training method for all models because it adjusts learning rates for individual parameters and delivers faster convergence than stochastic gradient descent. The training process started with a 1e-4 learning rate while using StepLR scheduler with step_size=5 and gamma=0.1 to decrease learning rates during training. The learning rate reduction mechanism through StepLR scheduler prevented overfitting from occurring during later training epochs. The implementation of early stopping used a validation Micro F1 score to determine when to stop training after three epochs of no improvement. Training stopped when performance reached a plateau and the best model weights were saved through checkpointing. The training duration for all models ranged between 15 to 20 epochs based on convergence patterns while early stopping automatically ended training after three epochs without validation Micro F1 improvement.

To prevent overfitting, dropout layers were used in all classifier heads (typically 0.5, or 0.25 in deeper heads). For data augmentation, we applied:

- Random resized crops
- Horizontal flips
- Color jittering (brightness, contrast, saturation)
- Normalization using ImageNet statistics

All images were resized to 224×224 for consistency and reduction of GPU usage across large models.

2.2 Threshold Tuning and Ensemble Strategy

Thresholding proved essential due to the dataset's label sparsity (i.e., most labels are 0). We implemented global threshold tuning, sweeping values from 0.05 to 0.95 during validation to find the best balance of precision and recall.

Later in the project, we introduced ensemble models, averaging predictions from multiple architectures. While we experimented with weighted averaging, our best-performing submission came from simple average predictions of ConvNeXt-Large, EfficientNetV2-L, and EfficientNetV2-XL, using a global threshold of 0.25. This combination yielded our highest public Micro F1 score.

3. COMPARATIVE EXPERIMENTS AND RESULTS

3.1 Overview of Submission Strategy

We conducted 43 total submissions during the competition, iteratively refining our models, thresholds, and ensemble techniques. Every submission was guided by a documented change, logged carefully to track outcomes and reasoning. Our experiments ranged from fixing architectural mistakes to testing increasingly powerful model families and ensemble configurations.

Below we are showcasing a breakdown of the most significant experiments and the impact of our decisions on the Micro F1 score on the Kaggle public leaderboard.

3.2 Key Experimental Results

Table 1. Summary of experimental changes and their corresponding Micro F1 scores on the public Kaggle leaderboard.

Change #	Model(s) Used	Strategy / Modification	Micro F1 Score
1	EfficientNetB0	Double sigmoid error with BCEWithLogitsLoss(). Resulted in stuck loss	0.00000
2	EfficientNetB0	Fixed sigmoid issue, model trains properly	0.20349
3	ResNet50	New model + added data augmentation	0.27687
4	ConvNeXt-Tiny	New model architecture	0.28948
5	ConvNeXt-Tiny	Global threshold tuning (0.15)	0.34879
6	ConvNeXt-Tiny	Tried per-class thresholds (performance dropped)	0.33956
7	ConvNeXt-Tiny + ResNet50	Simple average ensemble, best global threshold restored	0.40771
8	EfficientNetV2-M	Switched to EfficientNetV2 series	0.42495
9	EffNetV2-M + ResNet50	Weighted ensemble: 0.65 EffNetV2 + 0.35 ResNet	0.43890
10	EfficientNetV2-L	Best global threshold locked in 0.25	0.47388

11	EffNetV2-M + L	Avg preds, threshold 0.15 or 0.25	0.47954
12	EffNetV2-M + L + ResNet50	Triple ensemble	0.47619
13	EfficientNetV2-XL	Larger model, tuned threshold 0.20	0.45175
14	EffNetV2-L + XL	Multiple threshold tests from 0.15 to 0.25 (0.25 proved best)	0.48263
15	EffNetV2-M + L + XL	Larger ensemble, threshold 0.20	0.48839
16	ConvNeXt-Large	Powerful model, global threshold 0.30	0.46609
17	ConvNeXt-L + EffNetV2-XL	Avg preds, threshold 0.25	0.48228
18	ConvNeXt-L + EffNetV2-L	Tried avg preds (best) and weighted preds (0.65 convnext + 0.35 effnet)	0.50059
19	ConvNeXt-L + EffNetV2-L + XL	Simple average (BEST submission)	**0.50088**
20	ConvNeXt-L + EffNetV2-L + XL + M	Larger ensemble (no benefit)	0.48638

Figure 1. Progression of Micro F1 scores across selected experimental changes.

Our iterative improvements become clearer through the Micro F1 scores we plotted during our key experimental changes (see Figure 1). The graph shows an upward non-linear trend starting from 0.00000 because of architectural misconfiguration. The score increased substantially when we fixed the activation function and then continued to improve through the implementation of EfficientNetV2 and ConvNeXt architectures. The implementation of threshold tuning and ensemble methods produced significant performance gains which reduced the weaknesses of individual models. Our best score of 0.50088 emerged from a simple average ensemble of ConvNeXt-Large, EfficientNetV2-L and EfficientNetV2-XL which proved that well-aligned strategies produce cumulative advantages.

3.3 Analysis and Observations

Our experimental trajectory revealed several key findings, both from successful strategies and failed attempts. Initially, we encountered a major pitfall by incorrectly applying a sigmoid activation function on top of a model trained with BCEWithLogitsLoss(). This redundancy effectively flattened the loss curve and prevented learning altogether, resulting in a score of 0.00000 on our first submission. Once corrected, the model's ability to learn improved immediately, raising our F1 score to over 0.20 in the next iteration.

The architectural evolution of our models played a pivotal role in driving performance upward. Starting from simple baselines such as EfficientNetB0 and ResNet50, we quickly transitioned to more modern, higher-capacity networks. ConvNeXt-Tiny marked an early improvement, while the EfficientNetV2 family—particularly the "L" and "XL" variants—consistently yielded better results. ConvNeXt-Large, introduced in the later stages of the project, also proved to be a powerful model, producing some of our highest individual scores.

Among all model-related strategies, threshold tuning emerged as one of the most crucial components of success. The default classification threshold of 0.5 severely underperformed due to the sparsity of labels in the dataset—most images had relatively few positive labels. By experimenting with global threshold values ranging from 0.15 to 0.30, we were able to substantially improve both precision and recall. This tuning strategy became a standard component of our validation pipeline.

In parallel, ensembling techniques provided reliable performance gains across many of our submissions. Averaging the predictions of different models almost always led to improved results, with simple averaging generally outperforming weighted averaging. Our most successful submission, a combination of ConvNeXt-Large, EfficientNetV2-L, and EfficientNetV2-XL with a global threshold of 0.25, achieved a Micro F1 score of **0.50088**, the highest of the project. Interestingly, attempts to extend the ensemble with additional models such as EfficientNetV2-M or ResNet50 did not yield further improvements and sometimes slightly reduced performance. This suggests that including weaker or inconsistent models may introduce noise and dilute the strengths of top-performing networks.

Overall, the most effective practices combined strong backbone architectures, threshold tuning, and a focused

ensemble of well-generalizing models. Detailed logging, along with consistent validation and model diagnostics, allowed us to navigate failures early and iterate toward an optimal solution.

4. CONCLUSION

In this project, we explored the task of multi-label food classification using deep convolutional neural networks. Beginning with baseline models such as EfficientNetB0 and ResNet50, we progressively adopted more advanced architectures, including ConvNeXt and EfficientNetV2, while experimenting with various training strategies. Key contributions to performance included careful threshold tuning, dropout-based regularization, data augmentation, and efficient use of early stopping. Our most successful approach was a simple averaging ensemble of ConvNeXt-Large, EfficientNetV2-L, and EfficientNetV2-XL, which achieved a Micro F1 score of 0.50088 on the Kaggle public leaderboard. Through iterative experimentation and detailed logging, we identified the value of model diversity, proper thresholding, and streamlined training practices. This work highlights how architectural choices and ensemble strategies can significantly enhance performance in complex, real-world multi-label classification tasks.

REFERENCES

[1] AICrowd, "Food Recognition Benchmark 2022," https://www.aicrowd.com/challenges/food-recognition-benchmark-2022, Accessed May 2025.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[4] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[6] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.